05. Train-Test Split & Validation Strategy

Recommender evaluation differs from ordinary supervised classification.

Project: Movie Recommender System

Source archive supplied in this conversation: Movie-recommender-system-main.zip | Notebook: notebook.ipynb | App: app.py

What repository does

There is no train/test split, cross-validation, user interaction dataset or relevance label. The notebook builds vectors for all available movies, calculates the full pairwise similarity matrix and inspects a few nearest-neighbour outputs. This is an item-item content similarity lookup, not a predictive model trained against held-out user preferences.

Why a conventional split is absent

A content-based item similarity engine can compute a representation for every catalogue item without labelled user outcomes. However, a split is still needed to evaluate whether the ranking retrieves relevant items, and to test robustness to new or changed catalogue items.

Recommended offline evaluation

- Collect user interactions such as clicks, saves, ratings, completed watches or explicit relevance judgments.
- Create temporal train/test splits so interactions in the future are held out.
- For each test user or seed item, generate top-N recommendations using only information available at that point.
- Measure Precision@K, Recall@K, MAP@K, NDCG@K and coverage; add diversity/novelty metrics.
- Use human evaluation for content relevance and filter out unavailable or inappropriate items.
- Compare against simple baselines: most popular, same genre and random recommendations.

Content-based cold start

A new movie can be represented from its metadata even when it has no user interactions, which is an advantage over pure collaborative filtering. It still requires sufficient and accurate metadata. New users are not a problem for a title-to-title engine because the user selects a seed title, but a user-personalized system would need preference information.

Validation cautions

- Do not treat high cosine similarity as evidence of recommendation quality.
- Avoid evaluation leakage by not using future interactions to build user profiles or ranking features.
- Evaluate multiple seed movies and different genres/languages/popularity levels.
- Check catalogue coverage and whether recommendations over-concentrate on blockbusters or franchises.